Get-Set-Go

Documenting My GoLang's Learning Journey

Complete Roadmap · 7 Phases · Backend Development + Linux Development
From Zero to Production-Grade Go Development

Dev-X-Akshat · 2026

Overview & End Goals

This document is your complete, living reference for the Go learning journey — from first principles to production-grade systems. It covers every topic you need to master, flags the things that trip up developers from other languages (especially Node.js/TypeScript and C++), and explains why each project matters and what you will genuinely walk away knowing.

Why Go — for your specific goals

You are targeting HFT and fintech engineering roles (Optiver, IMC, Citadel, Alpaca) by end of 2026. Here is why Go fits that path perfectly alongside your C++ trading engine work:

Advantage	Relevance to You
Concurrency model	Goroutines and channels make concurrent network I/O trivial — critical for market data feeds, order routing, and WebSocket connections.
Performance profile	Near-C++ speed for I/O-bound workloads with a fraction of the complexity. Perfect for backend services that sit alongside a C++ core.
Industry adoption	Docker, Kubernetes, Caddy, CockroachDB, and many fintech internal tools are written in Go. Knowing it makes you immediately useful.
Tooling & deployment	Single static binary, tiny Docker images (<10 MB), cross-compilation built-in — ideal for Linux-first infrastructure.
Complements C++	Your trading engine will be C++ at the hot path. Go becomes the glue: REST APIs, admin dashboards, data pipelines, monitoring.

Your End Goals

Backend path (Phases 1–3, 6): Build production-ready REST APIs, auth services, WebSocket servers, and microservices with gRPC. These map directly to the backend components of any trading platform — position tracking, risk APIs, execution services.

Linux systems path (Phases 4–5): Write native Linux tooling: system monitors, TUI dashboards, file watchers, process managers. This deepens your OS knowledge — essential for HFT infrastructure roles on bare-metal Linux.

Portfolio output: By Phase 7, you will have 10+ shipped projects, a published CLI tool, a deployed VPS API, and potentially an open-source contribution — all Go, all demonstrable.

Combined with C++ engine: Your C++ trading engine handles the nanosecond-level matching and execution. Go handles the surrounding ecosystem: market data ingestion APIs, web dashboards, backtesting runners, deployment tooling.

Roadmap at a Glance

#	Phase	Duration	Core Focus
1	Go Fundamentals	2–3 weeks	Variables, structs, interfaces, error handling, packages

#	Phase	Duration	Core Focus
2	**Concurrency & Stdlib**	2–3 weeks	Goroutines, channels, context, net/http, JSON
3	**Backend Development**	4–6 weeks	REST APIs, PostgreSQL, JWT auth, testing, migrations
4	**Linux Systems Programming**	3–4 weeks	syscall, signals, fsnotify, /proc, cross-compilation
5	**CLI Tooling & TUI Apps**	3–4 weeks	cobra, viper, Bubbletea, Lipgloss, GoReleaser
6	**Advanced Backend & Patterns**	4–6 weeks	WebSockets, gRPC, Kafka/NATS, Redis, Docker, pprof
7	**Beast Mode — Build Something Real**	Ongoing	Container runtime, custom HTTP server, key-value DB, OSS contrib

Critical Awareness — What Go Developers Must Know

These are the most common pitfalls, mental model shifts, and Go-specific behaviours that trip up developers coming from Node.js/TypeScript and C++. Read this section before writing a single line of Go.

Go Philosophy — Embrace It, Not Fight It

- **gofmt is law:** There is no debate about formatting. Run gofmt (or goimports) on every save. Your editor should do this automatically. Non-formatted code is unacceptable in Go projects.
- **Explicit over implicit:** Go has no exceptions, no default arguments, no function overloading, no generics magic (until recently). Everything is explicit. This feels verbose at first but makes code trivially readable 6 months later.
- **Errors are values:** Unlike Node.js (throw/catch) or C++ (exceptions), Go returns errors as the second return value. NEVER ignore an error with _. Every error must be handled or explicitly propagated.
- **Short variable names are idiomatic:** i, n, r, w, ctx are normal in Go. Long names are for package-level identifiers. Short-scope variables should be short. Do not fight this.
- **Unused imports and variables are compile errors:** Go will not compile if you import a package you do not use, or declare a variable you do not use. This is a feature, not a bug.

Interfaces — Go's Most Powerful (and Misunderstood) Feature

- **Interfaces are implicit:** Unlike C++ (virtual) or TypeScript (implements), you never declare that a type implements an interface. If it has the methods, it satisfies the interface. This is structural typing.
- **Keep interfaces small:** The io.Reader interface has exactly one method. The io.Writer interface has exactly one method. The Go proverb: "The bigger the interface, the weaker the abstraction." Prefer many small interfaces.
- **Accept interfaces, return structs:** Functions should accept interface types as parameters (flexible) but return concrete types (clear). This is the most important design rule in Go.
- **nil interface vs nil pointer:** A nil pointer wrapped in an interface is NOT nil. This is the single most common bug for new Go developers. An interface value is nil only when both its type and value are nil.

Concurrency — Where Most Bugs Hide

- **Goroutines are not threads:** They are multiplexed onto OS threads by the Go scheduler. You can spawn a million of them. But leaking goroutines (forgetting to cancel/close them) will silently eat memory.
- **Always think about goroutine lifetime:** Every goroutine you spawn must have a clear exit condition. Use context.Context for cancellation. Use sync.WaitGroup to wait for completion. Never fire-and-forget without a shutdown path.

- **Channel direction matters:** Pass chan<- T for send-only and <-chan T for receive-only. This self-documents intent and prevents misuse at compile time.
- **select with a default is non-blocking:** select { case v := <-ch: ... default: } returns immediately if ch is empty. Without default, it blocks. Know the difference.
- **sync.Mutex vs channels:** Use channels to communicate between goroutines. Use Mutex when protecting shared state. Mixing them unnecessarily creates deadlocks.
- **The race detector is your friend:** Run go test -race on every concurrent package. Go's built-in race detector catches data races at runtime. Use it in CI, always.

Slices & Maps — Sneaky Memory Behaviour

- **Slices are references to arrays:** Assigning a slice copies the header (pointer + length + capacity), not the data. Modifying a slice in a function modifies the original backing array. Use copy() when you need independence.
- **Append may or may not reallocate:** If cap is sufficient, append reuses the backing array. If not, it allocates a new one and copies. Never rely on the original slice being untouched after an append.
- **Maps are nil by default:** var m map[string]int declares a nil map. Assigning to a nil map panics. Always initialize: m := make(map[string]int) or m := map[string]int{}.
- **Map iteration order is random:** By design and enforced by the runtime. Never rely on map ordering. If you need ordered output, sort the keys explicitly.
- **Deleting from a map during range is safe:** Unlike some languages, Go explicitly allows deleting map keys inside a range loop. Additions during range may or may not be visited.

Error Handling Patterns

- **Wrap errors with context:** Use fmt.Errorf("doing X: %w", err) to wrap errors with context while preserving the original. Use errors.Is() and errors.As() to unwrap and check.
- **Sentinel errors:** Package-level var ErrNotFound = errors.New("not found") are sentinel errors. Check with errors.Is(err, ErrNotFound). Common in standard library (io.EOF, sql.ErrNoRows).
- **Do not panic in library code:** panic is for truly unrecoverable programmer errors (nil pointer, impossible state). Public library functions must return errors, not panic.
- **defer for cleanup:** defer file.Close() runs when the function returns, even on error. Always defer cleanup immediately after opening a resource — do not defer it at the bottom of the function conditionally.

Coming from Node.js/TypeScript

- **No async/await — use goroutines:** Go's concurrency is not Promise-based. A goroutine is like launching an async function, but channels replace callbacks and Promises. The mental model is different — concurrency via communication, not async state machines.
- **No npm ecosystem (mostly):** Go has modules (go.mod). The standard library is extremely rich — you will reach for third-party packages far less than in Node.js. Prefer stdlib solutions.
- **Static typing is stricter:** No any escape hatch (unlike TypeScript's any). No implicit type coercion. The compiler rejects every type mismatch. This feels restrictive and then feels liberating.

- **No classes — use structs + methods:** Go has no inheritance. Composition via embedding is the pattern. A struct can embed another struct to inherit its methods. This is not the same as inheritance — understand the difference.

Coming from C++

- **No manual memory management:** Go has a garbage collector. You do not call free(). You also cannot control memory layout the way you can in C++. For latency-critical paths, minimize allocations and reuse buffers with sync.Pool.
- **No RAII:** C++ destructors run deterministically. In Go, use defer for cleanup. The GC reclaims memory, but defer handles all other cleanup (file closes, mutex unlocks, connection returns).
- **No header files, no forward declarations:** Go files in the same package see each other's identifiers. The package is the compilation unit, not the file.
- **Build constraints replace #ifdef:** Use //go:build linux or //go:build !windows for conditional compilation. These are comments at the top of the file, not preprocessor directives.

Phase 1 — Go Fundamentals

Phase 1 — **Go Fundamentals**
~2–3 weeks · the language itself

TOPICS TO COVER

- Variables & types
- Pointers
- Packages & modules
- Functions & closures
- Arrays, slices & maps
- defer, panic, recover
- Structs & interfaces
- Error handling

WHAT TO FOCUS ON & BE AWARE OF

- Go is opinionated. Run gofmt on every save. Use goimports to manage imports automatically.
- Understand the difference between value receivers and pointer receivers on methods immediately — it matters for mutability and interface satisfaction.
- Interfaces are implicit (structural typing). A type satisfies an interface by having the right methods — no "implements" keyword.
- Error handling is not optional. Get comfortable with the if err != nil { return err } pattern from day one.
- The zero value of every type is meaningful in Go. An int is 0, a string is "", a pointer is nil. Design with zero values in mind.
- Learn go mod init, go get, go tidy early — module management is fundamental.

PROJECTS — WHY THEY MATTER

CLI Todo App

Why it matters: File I/O is one of the most common real-world tasks. A CLI todo app forces you to use os.Args, file persistence, struct serialisation, and proper error handling all in one small codebase. It is the "Hello World" that actually teaches you something.

You will learn: File I/O with os package, encoding/json for persistence, structs as data models, os.Args and the flag package for CLI input, basic error handling patterns.

Unit Converter

Why it matters: Simple but forces you to think in Go's function and package model. You will naturally extract conversion logic into a separate package, discovering Go's package visibility rules (exported vs unexported identifiers).

You will learn: Writing reusable packages, exported vs unexported identifiers (capitalisation rule), switch statements, numeric type conversions, basic testing with the testing package.

Number Guessing Game

Why it matters: Ties together user input, randomness, loops, and error handling in an interactive context. It sounds trivial but teaches you how Go handles stdin and what "graceful error handling" looks like in a real user-facing loop.

You will learn: bufio.Scanner for input, math/rand for randomness, for loops without a while keyword, error wrapping, how Go's type system prevents you from accidentally mixing int and string.

TIP Focus on understanding Go's opinionated style — gofmt, short variable names, explicit errors. Do not fight the language; embrace it. Every annoyance you feel now is a future teammate thanking you for readable code.

Phase 2 — Concurrency & Standard Library

Phase 2 — **Concurrency & Standard Library**
~2–3 weeks · Go's superpower

TOPICS TO COVER

- Goroutines
- Channels (buffered & unbuffered)
- select statement
- sync.WaitGroup & Mutex
- context package
- net/http basics
- encoding/json
- os, io, bufio

WHAT TO FOCUS ON & BE AWARE OF

- The goroutine + channel model is Go's defining feature. Understand WHY it is different from threading: goroutines are cheap (2 KB stack, grows as needed) and multiplexed by the Go scheduler.
- Learn the "done channel" pattern and context cancellation before writing any production concurrent code. Leaked goroutines are silent killers.
- Use the race detector (go run -race or go test -race) from the very first concurrent program you write.
- Understand when to use a channel vs a mutex: channels for communication between goroutines, mutexes for protecting shared state.
- context.Context is the standard way to propagate deadlines, cancellation, and request-scoped values through your call tree. Learn it deeply — every Go backend API uses it.
- net/http in the standard library is powerful enough for production servers. Do not reach for a framework until you understand what the stdlib is doing.

PROJECTS — WHY THEY MATTER

🔥 Port Scanner

Why it matters: This is the definitive goroutine "aha moment" project. A naive sequential port scanner takes minutes. A concurrent version using a goroutine-per-port with a worker pool completes in milliseconds. The performance difference is visceral and unforgettable. This single project will permanently change how you think about I/O-bound concurrency.

You will learn: Goroutine fan-out patterns, buffered channels as work queues, WaitGroup for coordinated shutdown, net.DialTimeout for non-blocking network probes, worker pool pattern to bound concurrency, context for timeout propagation.

Weather CLI

Why it matters: Every real backend service makes HTTP calls. This project teaches you Go's HTTP client model, JSON decoding into structs, and handling real-world API errors (rate limits, network failures, invalid responses) — patterns you will use in every production service.

You will learn: http.Get and http.Client, json.Decoder vs json.Unmarshal, struct tags (json:"field_name"), the flag package for CLI flags, API key management via environment variables, http.Response.Body.Close() (and why defer is perfect for it).

Concurrent File Downloader

Why it matters: Downloads from multiple URLs simultaneously, merges results, and tracks progress. This is a complete real-world pattern: fan-out to N workers, collect results, handle partial failures. The progress tracking piece forces you to coordinate between goroutines safely.

You will learn: sync.WaitGroup for coordinating N goroutines, channels for result collection, io.Copy for efficient streaming, atomic counters, graceful error aggregation (not just panicking on first error), buffered file I/O.

TIP The port scanner will change how you think about concurrency. Build it before moving on — it is that important. Run it sequentially first, measure the time, then rewrite it with goroutines and measure again.

Phase 3 — Backend Development

Phase 3 — **Backend Development**
~4–6 weeks · REST APIs, databases, auth

TOPICS TO COVER

- net/http deep dive
- Routing (Chi or stdlib)
- Middleware
- database/sql + sqlx
- PostgreSQL / SQLite
- JWT auth
- Testing (table-driven tests)
- Migrations (goose)

WHAT TO FOCUS ON & BE AWARE OF

- Start with stdlib net/http only. No Gin, no Echo, no Fiber. Build the pain point manually first so you understand what a framework is actually saving you from.
- Table-driven tests are Go's idiomatic testing pattern. Learn them early. They make it trivially easy to add edge cases to tests later.
- Understand the middleware chain pattern deeply: each middleware is a function that wraps an http.Handler. This is the foundation of every Go web framework.
- database/sql is the standard interface. sqlx extends it with useful helpers (StructScan, Named queries). Learn the difference between sql.Row and sql.Rows and always close rows.
- JWT auth: understand what the token contains (claims), how to sign and verify, and why refresh tokens exist. Do not store sensitive data in JWT claims — they are base64 encoded, not encrypted.
- Run goose migrations from your app startup or a separate migration binary — never apply raw SQL manually in production.

PROJECTS — WHY THEY MATTER

REST API — No Framework

Why it matters: Building a REST API with only the standard library forces you to understand routing, request parsing, response encoding, and middleware from first principles. When you later use Chi or any framework, you will understand exactly what it is doing and why. This is foundational muscle memory that separates Go developers who can debug anything from those who are helpless without their framework.

You will learn: http.ServeMux routing, request body decoding, response writing, HTTP status codes, middleware pattern, handler functions vs handler types, graceful shutdown with context, structured error responses.

URL Shortener with PostgreSQL

Why it matters: A URL shortener is deceptively complete: it needs a database, unique ID generation, redirect logic, and hit analytics. Building it forces you through the entire database integration lifecycle — schema design, migrations, CRUD, and reading back data — in a realistic context.

You will learn: database/sql and sqlx, PostgreSQL driver (pgx or lib/pq), goose migrations, CRUD operations, redirect handling (http.Redirect), basic analytics with SQL aggregations, environment-based config.

Auth Service (JWT + Refresh Tokens)

Why it matters: Authentication is in every production service, and doing it correctly requires understanding cryptographic hashing, JWT structure and lifetime, refresh token rotation, and secure middleware. Building this yourself means you will never blindly trust a third-party auth library you cannot debug.

You will learn: bcrypt for password hashing, JWT signing and verification (golang-jwt/jwt), access vs refresh token lifecycle, middleware for protected routes, secure cookie/header patterns, token revocation with a database blocklist.

Start with stdlib only — resist Gin/Echo until you feel the pain of manual routing. You will appreciate frameworks more and understand what they save you. Then pick Chi — it is stdlib-compatible and used in production at major companies.

Phase 4 — Linux Systems Programming

Phase 4 — **Linux Systems Programming**
~3–4 weeks · OS, files, processes

TOPICS TO COVER

- syscall package
- os/exec — run commands
- File system operations
- Signals (SIGTERM, SIGINT)
- stdin/stdout/stderr pipes
- inotify / fsnotify
- Cross-compilation
- Embedding files (go:embed)

WHAT TO FOCUS ON & BE AWARE OF

- This phase is uniquely powerful on your Ubuntu setup (Intel Core 7 240H). Go's Linux-native tooling means you can interact with /proc, inotify, and cgroups directly from Go code.
- Signal handling is critical for production services. Every long-running process must handle SIGTERM for graceful shutdown. Learn the os/signal package pattern.
- Go's cross-compilation is trivial: GOOS=linux GOARCH=arm64 go build. Understand CGO_ENABLED=0 for fully static binaries — essential for scratch Docker images.
- go:embed is underrated. Embedding static files, SQL migrations, or templates directly into your binary makes deployment trivial — one file, no external dependencies.
- Reading /proc files (meminfo, stat, net/dev) in Go is just file I/O. This is how production monitoring tools work. It feels like magic the first time.

PROJECTS — WHY THEY MATTER

Build Your Own grep / ls

Why it matters: Reimplementing Unix tools teaches the Unix philosophy from the inside: everything is a stream of bytes, programs should read stdin and write stdout, and composability (pipes) is the architecture. You will understand why Go's io.Reader and io.Writer interfaces exist and mirror the Unix model exactly.

You will learn: os.Walk for directory traversal, regexp package for pattern matching, stdin piping (os.Stdin is an io.Reader), flag package for options, bufio.Scanner for line-by-line reading, os.Stdout/Stderr for POSIX-correct output routing.

File Organizer / Watcher

Why it matters: A file watcher that auto-organises files demonstrates real-world event-driven programming with OS-level events. This is how development tools (hot reload, build systems) work. On Linux specifically, this uses inotify under the hood — understanding this makes you a stronger systems developer.

You will learn: fsnotify for cross-platform file watching, goroutines for concurrent event processing, time.AfterFunc for debouncing rapid events, os.Rename for atomic file moves, path/filepath for platform-safe path manipulation.

System Monitor (htop lite)

Why it matters: Reading /proc/stat, /proc/meminfo, and /proc/net/dev to build a live terminal system monitor is a rite of passage on Linux. This project synthesises everything: goroutines for polling, channels for update events, /proc parsing, and terminal rendering — all without any external library dependencies.

You will learn: Reading and parsing /proc files, ticker-based polling with time.Ticker, atomic operations for thread-safe counters, terminal ANSI escape sequences for in-place updates (cursor movement, clearing lines), percentage calculations from raw kernel counters.

💡 **TIP**

Reading /proc/meminfo and /proc/stat in Go feels like superpower-level access to the OS — and it is just file I/O. Linux exposes everything as files and Go handles files beautifully. This phase will make you a stronger Linux developer regardless of what language you use day-to-day.

Phase 5 — CLI Tooling & TUI Apps

Phase 5 — **CLI Tooling & TUI Apps**
~3–4 weeks · professional native tools

TOPICS TO COVER

- cobra (CLI framework)
- viper (config management)
- Bubbletea (TUI framework)
- Lipgloss (terminal styling)
- Terminal raw mode
- ANSI escape codes
- Shell completion
- Releasing with GoReleaser

WHAT TO FOCUS ON & BE AWARE OF

- Bubbletea uses the Elm Architecture (Model-Update-View). Understand the pattern before writing any TUI code. Once it clicks, building complex terminal UIs becomes systematic rather than ad hoc.
- cobra + viper is the industry standard for Go CLIs. Docker, Kubernetes, Hugo, and GitHub CLI all use cobra. Learn the subcommand pattern well.
- Shell completion (bash, zsh, fish) is a first-class feature in cobra. It is a one-liner to generate. Always ship it — it is what separates professional tools from scripts.
- GoReleaser automates cross-platform builds, .deb packages, Homebrew taps, and GitHub Releases from a single config file. Learn it once, use it forever.
- Lipgloss is to terminal styling what CSS is to web. Define styles once, apply everywhere. Combine with Bubbletea for production-quality TUI apps.

PROJECTS — WHY THEY MATTER

🔥 TUI Dashboard (Bubbletea)

Why it matters: A full terminal application with multiple panes, keyboard navigation, and live data updates is the most impressive thing you can build with Go on Linux. This is what Charm's ecosystem was designed for, and it is what makes Go on Linux so viscerally satisfying. Combined with your system monitor from Phase 4, this becomes a real tool you will actually use.

You will learn: Bubbletea Model-Update-View architecture, tea.Cmd for I/O and async operations, Lipgloss for component styling, terminal viewport management, keyboard event handling, live data updates via ticker commands, multi-pane layouts.

Git Helper CLI (cobra)

Why it matters: Building a real CLI tool with subcommands, config file support, and shell completions teaches you the professional CLI development workflow. Git helper is perfect because you use git daily, so you will actually use the tool. This project is also a strong portfolio piece — it is

immediately understandable to any developer who reviews it.

You will learn: cobra command tree and subcommand pattern, viper for .githelper.yaml config file, persistent flags vs local flags, shell completion generation, os/exec for running git commands, man page generation with cobra-man.

Publish a Real CLI Tool

Why it matters: Shipping is the most important skill. Publishing a Go CLI tool to GitHub with automated releases is the capstone of this phase. GoReleaser handles cross-compilation and packaging. Once published, anyone on macOS or Linux can install your tool with a single command. This is a portfolio milestone: a publicly available tool at a GitHub URL is worth more than ten "hello world" repositories.

You will learn: GoRereleaser configuration (.goreleaser.yml), GitHub Actions for automated release pipeline, cross-compilation for linux/amd64, linux/arm64, darwin/amd64, darwin/arm64, windows/amd64, .deb package generation, Homebrew tap creation, semantic versioning with git tags.

💡 **TIP**

Bubbletea by Charm is what Docker and Kubernetes CLI teams use for TUI applications. It is the production standard — learn it and you can build anything terminal-based. Start with the official Bubbletea tutorials before jumping to the dashboard project.

Phase 6 — Advanced Backend & Real-World Patterns

Phase 6	Advanced Backend & Real-World Patterns ~4–6 weeks · production-grade Go

TOPICS TO COVER

- WebSockets
- gRPC + protobuf
- Message queues (Kafka / NATS)
- Redis caching
- Docker (scratch images)
- Structured logging (slog)
- Profiling (pprof)
- OpenTelemetry

WHAT TO FOCUS ON & BE AWARE OF

- WebSockets in Go: one goroutine per connection reading, one writing. Never read and write from the same goroutine. Use a hub struct with a broadcast channel to fan out messages.
- gRPC + protobuf: define your service in a .proto file first. Generate the Go code. Never write gRPC server/client code by hand. Understand why protobuf is faster and smaller than JSON.
- pprof is built into the Go runtime. Expose a /debug/pprof endpoint in development and use go tool pprof to visualise heap allocations, goroutine counts, and CPU profiles. This is how you find the 20% of code causing 80% of allocations.
- slog (structured logging, added in Go 1.21) is now the standard. Use it instead of logrus or zap for new projects. JSON output for production, text for development.
- Scratch Docker images: GOOS=linux CGO_ENABLED=0 go build produces a fully static binary with no external dependencies. FROM scratch + COPY binary /binary = a Docker image under 10 MB. This is a competitive advantage in deployment cost and attack surface.
- OpenTelemetry is the standard for distributed tracing. Set it up once and your entire request lifecycle becomes observable across services.

PROJECTS — WHY THEY MATTER

🔥 Real-time Chat (WebSockets)

Why it matters: A WebSocket chat server is the canonical demonstration of Go's concurrency model at scale. Each connection runs two goroutines (reader + writer) and communicates through a central hub via channels. This is a textbook example of the "share memory by communicating" principle. At 10,000 concurrent connections, Go will use ~200 MB of memory. The same server in Node.js would use 2–3 GB. This difference matters in every fintech context.

You will learn: gorilla/websocket or nhooyr.io/websocket, hub pattern with broadcast channel, per-connection goroutine lifecycle, graceful connection cleanup, room-based message routing, JSON message envelope pattern, concurrent connection counters with atomic.

Microservice with gRPC

Why it matters: gRPC is the inter-service communication standard in HFT and fintech infrastructure. Every serious multi-service backend (pricing service, risk service, execution service) uses gRPC for internal communication because it is typed, fast, and generates client code automatically. Understanding how to define a protobuf schema, generate Go code from it, and implement both server and client is table-stakes for senior backend roles.

You will learn: Protocol Buffers syntax (.proto files), protoc code generation, grpc-go server implementation, grpc-go client with connection pooling, unary vs streaming RPCs, interceptors (gRPC middleware), service reflection for debugging, error handling with status codes.

Deploy a Go API on a VPS

Why it matters: Deploying your own service on a real VPS (DigitalOcean, Hetzner, Vultr) with a scratch Docker image, systemd service, and nginx reverse proxy is the capstone of backend Go. This is the full Linux stack in production. At this point you have written the API, tested it, containerised it to under 10 MB, deployed it, and made it accessible via HTTPS. This is what production looks like.

You will learn: Multi-stage Dockerfile (builder + scratch), systemd unit file for process management, nginx reverse proxy config, Let's Encrypt TLS with certbot, environment variable management in production, log aggregation with journalctl, zero-downtime restart with systemd.

TIP

Deploy your chat app to a VPS with a 5 MB Docker image and watch it handle 10,000 concurrent connections on a $6/month server. That is the moment you become a Go believer for life — and a compelling story to tell in any engineering interview.

Phase 7 — Beast Mode — Build Something Real

Phase 7 — **Beast Mode — Build Something Real**
Ongoing · your magnum opus

TOPICS TO COVER

- Linux namespaces & cgroups
- Raw TCP/socket programming
- WAL logging & file persistence
- Open source contribution workflow
- Benchmarking & profiling at scale
- Systems design patterns

WHAT TO FOCUS ON & BE AWARE OF

- Pick exactly one Phase 7 project. The power comes from going deep, not wide. A half-finished container runtime teaches you nothing. A finished one teaches you everything.
- At this level, read the source code of the projects you use. Read the Go standard library source. Read k9s, fzf, and Hugo's internals. You are a Go developer now — start thinking like one.
- Benchmark everything with go test -bench. Understand the benchstat tool for statistical significance. Know your allocations per operation (B/op and allocs/op). This is the language of HFT engineering.
- Open source contribution: find a small issue in a project you actually use, fix it, and submit a PR. The code review from a maintainer of a serious Go project is worth more than any tutorial.

PROJECTS — WHY THEY MATTER

Mini Container Runtime (Docker lite)

Why it matters: Building a container runtime from scratch — even a minimal one — teaches you more about Linux than any other project. You will use namespaces (PID, network, mount, UTS) and cgroups to create genuine process isolation. This is the deepest OS knowledge available from userspace. Interviewers at infrastructure companies are genuinely impressed by candidates who have done this.

You will learn: Linux namespaces (CLONE_NEWPID, CLONE_NEWNS, CLONE_NEWNET), cgroups v2 for resource limits, chroot and pivot_root for filesystem isolation, syscall package for low-level Linux calls, /proc/self/exe trick for re-execution, overlayfs for layered filesystems, seccomp for syscall filtering.

Build Your Own HTTP Server from TCP

Why it matters: Implementing HTTP/1.1 on top of raw TCP forces you to understand exactly what every HTTP framework is doing. You will parse request lines, headers, bodies, implement keep-alive connection reuse, and handle chunked transfer encoding. After this, HTTP is never a black box

again. This is a classic SWE interview topic at systems-level companies.

You will learn: net.Listen and net.Conn, manual HTTP/1.1 request parsing, header normalisation, body framing (Content-Length vs chunked), keep-alive and connection pooling, concurrent request handling, proper connection lifecycle management, HTTP response formatting.

Simple Key-Value Database

Why it matters: Building a persistent key-value store with a write-ahead log (WAL) teaches you how every real database works under the hood: how writes are made durable before being applied, how recovery works after a crash, and why B-trees exist. This directly applies to understanding storage engines in the fintech databases you will work with (RocksDB, LMDB, Redis persistence).

You will learn: Write-ahead log (WAL) for crash recovery, memory-mapped files (mmap via syscall), index data structures (hashmap + B-tree concepts), file locking for single-writer safety, binary encoding for space efficiency, compaction for reclaiming space from deleted keys, a basic query/command parser.

Contribute to a Real Go Project

Why it matters: A merged pull request to a serious open-source Go project (Hugo, Caddy, k9s, Bubbletea, fzf) is a permanent, public, verifiable demonstration of your Go competency. It also teaches the single most important professional skill: reading unfamiliar codebases quickly. The feedback from an experienced maintainer is irreplaceable.

You will learn: Reading and navigating large Go codebases, Go code review culture and idioms, writing tests that satisfy a senior maintainer, submitting and iterating on PRs, understanding contribution guidelines and CI pipelines, git workflow in a collaborative open-source project.

💡 **TIP** At this point you are not learning Go anymore — you are just a Go developer. Welcome to the guild. The gopher mascot exists because the language is genuinely fun to write. You will understand why now.

Resources & Next Steps

Essential Go Resources

tour.golang.org	The official Go tour — complete this before anything else in Phase 1. Interactive and authoritative.
go.dev/doc/effective_go	Effective Go — the canonical style and idiom guide. Read it once in Phase 1 and re-read after Phase 3.
gobyexample.com	Concise, runnable examples of every Go concept. Your quick-reference for the entire journey.
100 Go Mistakes (book)	Teiva Harsanyi. The single best book for avoiding the pitfalls covered in this document. Read alongside Phase 3+.
Concurrency in Go (book)	Katherine Cox-Buday. The definitive deep dive into goroutines, channels, and the Go memory model. Essential for Phase 2.
charm.sh	Official Charm documentation for Bubbletea, Lipgloss, and the full Charm CLI ecosystem. Phase 5 home base.
pkg.go.dev	Official Go package documentation. Learn to read stdlib source code here — it is the best Go code available.
r/golang + Gopher Slack	Community. Ask questions, read discussions, follow Go proposals. The Go community is genuinely helpful.

Connecting Go to Your C++ Trading Engine

1. Phase 2 (concurrency) directly prepares you to build the Go market data ingestion layer that feeds your C++ engine — WebSocket streams, Alpaca market data, order book aggregation.
2. Phase 3 (backend) enables the REST API layer that sits in front of your engine — position endpoints, risk dashboards, backtesting submission APIs.
3. Phase 6 (gRPC) is how your Go services will communicate with the C++ core — protobuf-defined interfaces, zero-copy where possible, streaming for market data.
4. Phase 4 + 7 (Linux systems) will deepen your understanding of process pinning, CPU affinity, and memory-mapped files — all relevant to optimising the C++ engine itself.

By the end of this roadmap you will have shipped 10+ Go projects, deployed a real API to production, published at least one CLI tool, and potentially contributed to an open-source Go codebase. That is a portfolio that speaks louder than a resume bullet saying "familiar with Go." You will be a Go developer — not someone who has watched tutorials about Go.